

1. Introduction to Operating Systems

Without its software, a computer is basically a useless lump of metal. With its software, a computer can store, process, and retrieve information; play music and videos; send e-mail, search the Internet; and engage in many other valuable activities to earn its keep.

A modern computer system consists of one or more processors, some main memory, disks, printers, a keyboard, a display, network interfaces, and other input/output devices - all in all, a complex system. Writing programs that keep track of all these components and use them correctly, let alone optimally, is an extremely difficult job. If every programmer had to be concerned with how disk drives work, and with all the dozens of things that could go wrong when reading a disk block, it is unlikely that many programs could be written at all.

Long ago, it became abundantly clear that some way had to be found to shield programmers from the complexity of the hardware. The way that has evolved gradually is to put a layer of software on top of the bare hardware, to manage all parts of the system, and present the user with an interface or virtual machine that is easier to understand and program. This layer of software is the *operating system*.

The placement of the operating system is shown in Figure 1. At the bottom is the hardware, which, in many cases, is itself composed of two or more levels (or layers). The lowest level contains physical devices, consisting of integrated circuit chips, wires, power supplies, cathode ray tubes, and similar physical devices.

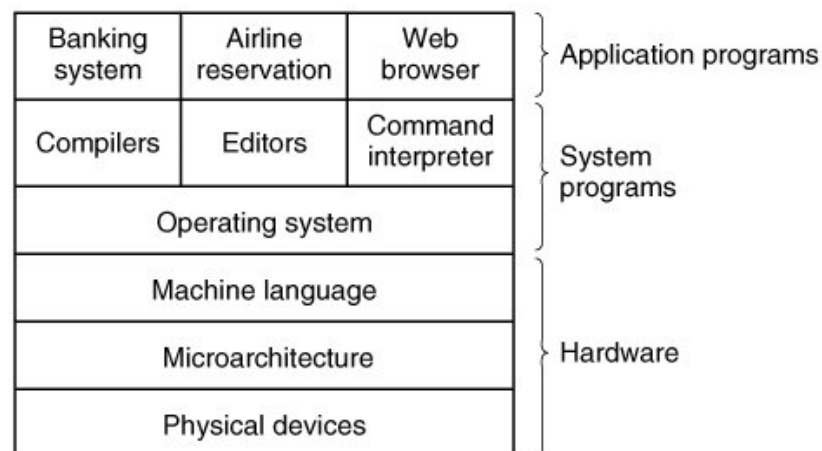


Fig. 1 A computer system consists of hardware, system programs, and application programs in the shown hierarchy.

Next comes the microarchitecture level, in which the physical devices are grouped together to form functional units. Typically this level contains some registers internal to the CPU (Central Processing Unit) and a data path containing an arithmetic logic unit. The purpose of the data path is to execute some set of instructions. These instructions may use registers or other hardware

facilities. Together, the hardware and instructions visible to an assembly language programmer form the ISA (Instruction Set Architecture) This level is often called machine language.

The machine language typically has between 50 and 300 instructions, mostly for moving data around the machine, doing arithmetic, and comparing values. In this level, the input/output devices are controlled by loading values into special device registers. For example, a disk can be commanded to read by loading the values of the disk address, main memory address, byte count, and direction (read or write) into its registers.

Following the aforementioned levels is the operating system. A major function of the operating system is to hide all this complexity and give the programmer a more convenient set of instructions to work with. For example, `read block from file` is conceptually much simpler than having to worry about the details of moving disk heads, waiting for them to settle down, and so on.

On top of the operating system is the rest of the system software. Here we find the command interpreter (shell), window systems, compilers, editors, and similar application-independent programs. It is important to realize that these programs are definitely not part of the operating system.

Finally, above the system programs come the application programs. These programs are purchased (or written by) the users to solve their particular problems, such as word processing, spreadsheets, engineering calculations, or storing information in a database.

What Is an Operating System?

Usually, it is difficult to pin down precisely what an operating system is. Part of the problem is that operating systems perform two basically unrelated functions, *extending the machine* and *managing resources*, and depending on who is doing the talking, you hear mostly about one function or the other.

The Operating System as an Extended Machine

As mentioned earlier, the architecture (instruction set, memory organization, I/O, and bus structure) of most computers at the machine language level is primitive and awkward to program, especially for input/output. To make this point more concrete, let us briefly look at how floppy disk I/O is done using the NEC PD765 compatible controller chips used on many Intel-based personal computers.

- The PD765 has 16 commands, each specified by loading between 1 and 9 bytes into a device register. These commands are for reading and writing data, moving the disk arm, and formatting tracks, as well as initializing, sensing, resetting, and recalibrating the controller and the drives.
- The most basic commands are read and write, each of which requires 13 parameters, packed into 9 bytes. These parameters specify such items as the address of the disk block to be read, the number of sectors per track, the recording mode used on the physical medium, the inter-sector gap spacing, and what to do with a deleted-data-address-mark.
- When the operation is completed, the controller chip returns 23 status and error fields packed into 7 bytes.
- As if this were not enough, the floppy disk programmer must also be constantly aware of whether the motor is on or off. If the motor is off, it must be turned on (with a long startup delay) before

data can be read or written. The motor cannot be left on too long, however, or the floppy disk will wear out. The programmer is thus forced to deal with the trade-off between long startup delays versus wearing out floppy disks (and losing the data on them).

Without going into the real details, it should be clear that the average programmer probably does not want to get too intimately involved with the programming of floppy disks (or hard disks, which are just as complex and quite different). Instead, what the programmer wants is a simple, high-level abstraction to deal with. In the case of disks, a typical abstraction would be that the disk contains a collection of named files. Each file can be opened for reading or writing, then read or written, and finally closed. Details such as whether or not recording should use modified frequency modulation and what the current state of the motor is should not appear in the abstraction presented to the user.

The program that hides the truth about the hardware from the programmer and presents a nice, simple view of named files that can be read and written is, of course, the operating system. Just as the operating system shields the programmer from the disk hardware and presents a simple file-oriented interface, it also conceals a lot of unpleasant business concerning interrupts, timers, memory management, and other low-level features. In each case, the abstraction offered by the operating system is simpler and easier to use than that offered by the underlying hardware.

In this view, the function of the operating system is to present the user with the equivalent of an extended machine or virtual machine that is easier to program than the underlying hardware. The operating system provides a variety of services that programs can obtain using special instructions called *system calls*.

The Operating System as a Resource Manager

The concept of the operating system as primarily providing its users with a convenient interface is a top-down view. An alternative, bottom-up, view holds that the operating system is there to manage all the pieces of a complex system. Modern computers consist of processors, memories, timers, disks, mice, network interfaces, printers, and a wide variety of other devices. In the alternative view, the job of the operating system is to provide for an orderly and controlled allocation of the processors, memories, and I/O devices among the various programs competing for them.

Imagine what would happen if three programs running on some computer all tried to print their output simultaneously on the same printer. The first few lines of printout might be from program 1, the next few from program 2, then some from program 3, and so forth. The result would be chaos. The operating system can bring order to the potential chaos by buffering all the output destined for the printer on the disk. When one program is finished, the operating system can then copy its output from the disk file where it has been stored to the printer, while at the same time the other program can continue generating more output, oblivious to the fact that the output is not really going to the printer (yet).

When a computer (or network) has multiple users, the need for managing and protecting the memory, I/O devices, and other resources is even greater, since the users might otherwise interfere with one another. In addition, users often need to share not only hardware, but information (files, databases, etc.) as well. In short, this view of the operating system holds that its primary task is to

- *keep track of who is using which resource*
- *grant resource requests*
- *account for usage and*
- *mediate conflicting requests from different programs and users.*

Resource management includes multiplexing (sharing) resources in two ways: in time and in space. When a resource is time multiplexed, different programs or users take turns using it. First one of them gets to use the resource, then another, and so on. For example, with only one CPU and multiple programs that want to run on it, the operating system first allocates the CPU to one program, then after it has run long enough, another one gets to use the CPU, then another, and then eventually the first one again. *Determining how the resource is time multiplexed who goes next and for how long is the task of the operating system.*

Another example of time multiplexing is sharing the printer. When multiple print jobs are queued up for printing on a single printer, a decision has to be made about which one is to be printed next.

The other kind of multiplexing is space multiplexing. Instead of the customers taking turns, each one gets part of the resource. For example, main memory is normally divided up among several running programs, so each one can be resident at the same time (for example, in order to take turns using the CPU). Assuming there is enough memory to hold multiple programs, it is more efficient to hold several programs in memory at once rather than give one of them all of it, especially if it only needs a small fraction of the total.

Another resource that is space multiplexed is the (hard) disk. In many systems a single disk can hold files from many users at the same time. Allocating disk space and keeping track of who is using which disk blocks is a typical operating system resource management task.

History of Operating Systems

Operating systems have been evolving through the years. Since operating systems have historically been closely tied to the architecture of the computers on which they run, we will look at successive generations of computers to see what their operating systems were like. This mapping of operating system generations to computer generations is crude, but it does provide some structure where there would otherwise be none.

The First Generation (1945 - 55) : *Vacuum Tubes and Plug boards*

The first true digital computer was designed by the English mathematician Charles Babbage (1792 - 1871). Although Babbage spent most of his life and fortune trying to build his "analytical engine," he never got it working properly because it was purely mechanical, and the technology of his day could not produce the required wheels, gears, and cogs to the high precision that he needed.

After Babbage's unsuccessful efforts, little progress was made in constructing digital computers until World War II. Around the mid-1940s, Howard Aiken at Harvard University, John von Neumann at the Institute for Advanced Study in Princeton, J. Presper Eckert and John Mauchley at the University of Pennsylvania, and Konrad Zuse in Germany, among others, all succeeded in building calculating engines. The first ones used mechanical relays but were very slow, with cycle times measured in seconds. Relays were later replaced by vacuum tubes.

In these early days,

- A single group of people designed, built, programmed, operated, and maintained each machine.
- All programming was done in absolute machine language, often by wiring up plug boards to control the machine's basic functions.
- Programming languages were unknown (even assembly language was unknown).
- Operating systems were unheard of. The usual mode of operation was for the programmer to sign up for a block of time on the signup sheet on the wall, then come down to the machine room, insert his or her plug board into the computer, and spend the next few hours hoping that none of the 20,000 or so vacuum tubes would burn out during the run.
- Virtually all the problems were straightforward numerical calculations, such as grinding out tables of sines, cosines, and logarithms.

By the early 1950s, the routine had improved somewhat with the introduction of punched cards. It was now possible to write programs on cards and read them in instead of using plug boards-otherwise, the procedure was the same..

The Second Generation (1955 - 65): *Transistors and Batch Systems*

The introduction of the transistor in the mid-1950s changed the picture radically. Computers became reliable enough that they could be manufactured and sold to paying customers with the expectation that they would continue to function long enough to get some useful work done. For the first time, there was a clear separation between designers, builders, operators, programmers, and maintenance personnel.

These machines, now called *mainframes*, were locked away in specially air conditioned computer rooms, with staffs of specially-trained professional operators to run them. To run a job (i.e., a program or set of programs), a programmer would first write the program on paper (in FORTRAN or possibly even in assembly language), then punch it on cards. He would then bring the card deck down to the input room and hand it to one of the operators and go drink coffee until the output was ready.

When the computer finished whatever job it was currently running, an operator would go over to the printer and tear off the output and carry it to the programmer/user. Then he would take one of the card decks and read it in. If the FORTRAN compiler was needed, the operator would have to get it from a file cabinet and read it in. Much computer time was wasted while operators were walking around the machine room.

Given the high cost of the equipment, it is not surprising that people quickly looked for ways to reduce the wasted time. The solution generally adopted was the batch system. *The idea behind it was to collect a tray full of jobs in the input room and then read them onto the processing machine all at once.* Described as simple steps, the situation in those days is:

- After about an hour of collecting a batch of jobs, the tape was rewound and brought into the machine room, where it was mounted on a tape drive.
- The operator then loaded a special program (the ancestor of today's operating system), which read the first job from tape and ran it. The output was written onto a second tape, instead of being printed.
- After each job finished, the operating system automatically read the next job from the tape and began running it.
- When the whole batch was done, the operator removed the input and output tapes, replaced the input tape with the next batch, and sent the output tape for printing

Large second-generation computers were used mostly for scientific and engineering calculations, such as solving the partial differential equations that often occur in physics and engineering. They were largely programmed in FORTRAN and assembly language. Typical operating systems were FMS (the Fortran Monitor System) and IBSYS, IBM's operating system for the 7094.

The Third Generation (1965-1980) : ICs and Multiprogramming

By the early 1960s, most computer manufacturers had two distinct, and totally incompatible, product lines. On the one hand there were the word-oriented, large-scale scientific computers, such as the 7094, which were used for numerical calculations in science and engineering. On the other hand, there were the character-oriented, commercial computers, such as the 1401, which were widely used for tape sorting and printing by banks and insurance companies.

But in quest of more power and versatility, IBM introduced the System/360. The 360 was designed to handle both scientific (i.e., numerical) and commercial computing. Thus a single family of machines could satisfy the needs of all customers. The 360 was also the first major computer line to use (small-scale) Integrated Circuits (ICs), thus providing a major price/performance advantage over the second-generation machines, which were built up from individual transistors

IBM has come up with an operating system called OS/360 for its 360 line¹. OS/360 and the similar third-generation operating systems produced by other computer manufacturers popularized several key techniques absent in second-generation operating systems. Probably the most important of these was ***multiprogramming*** – *a technique that increases the CPU usage by keeping as many jobs as possible in the memory.*

On the older systems like 7094, when the current job paused to wait for a tape or other I/O operation to complete, the CPU simply sat idle until the I/O finished. With heavily CPU-bound scientific calculations, I/O is infrequent, so this wasted time is not significant. With commercial

¹ The 370, 4300, 3080, 3090, and Z series, which used more modern technology than 360

data processing, the I/O wait time can often be 80 or 90 percent of the total time, so something had to be done to avoid having the (expensive) CPU be idle so much.

The solution that evolved was to partition memory into several pieces, with a different job in each partition, as shown in Figure 2. While one job was waiting for I/O to complete, another job could be using the CPU. If enough jobs could be held in main memory at once, the CPU could be kept busy nearly 100 percent of the time. Having multiple jobs safely in memory at once requires special hardware to protect each job against snooping and mischief by the other ones, but the 360 and other third-generation systems were equipped with this hardware.

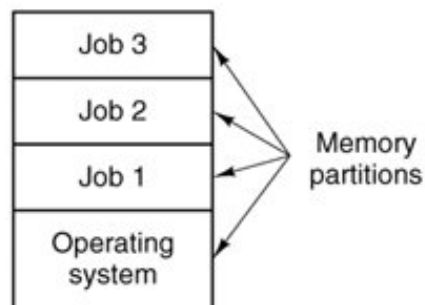


Figure 2 A multiprogramming system with three jobs in memory

Another major feature present in third-generation operating systems was the ability to read jobs *from cards onto the disk* as soon as they were brought to the computer room. Then, whenever a running job finished, the operating system could load a new job from the disk into the now-empty partition and run it. This technique is called **spooling** (from Simultaneous Peripheral Operation On Line) and was also used for output.

As improvements kept on going, operating systems with a variant of multiprogramming feature, called **timesharing**, were introduced. In timesharing systems many users communicate with the central system with their own online terminal. This enables the computer to provide fast, interactive service to a number of users simultaneously by giving each user a slice of the CPU time (in fact each user is given the impression that she has her own computer, whereas actually one computer is being shared among many users). By the way, multiprogramming and time sharing are the central themes of modern operating systems.

Time-sharing operating systems are more complex than multiprogramming operating systems. In these systems

- As in multiprogramming, several jobs must be kept simultaneously in memory, which requires some form of memory management and protection.
- So that a reasonable response time can be obtained, jobs may have to be swapped in and out of main memory to the disk that now serves as a backing store for main memory. A common method for achieving this goal is known as **virtual memory**, which is a technique that allows the execution of a job that may not be completely in memory.
- A mechanism for concurrent execution, which requires sophisticated scheduling schemes, must be provided.

- Mechanisms for job synchronization and communication must be provided to ensure orderly execution of the various jobs

The first serious timesharing system, CTSS (Compatible Time Sharing System), was developed at M.I.T. on a specially modified 7094. After the success of the CTSS system, MIT, Bell Labs, and General Electric decided to embark on the development of a "computer utility," a machine that would support hundreds of simultaneous timesharing users. This system, known as MULTICS (MULTiplexed Information and Computing Service) was designed to support hundreds of users on a machine only slightly more powerful than an Intel 80386-based PC, although it had much more I/O capacity.

Another major development during the third generation was the phenomenal growth of minicomputers, starting with the Digital Equipment Company (DEC) PDP-1 in 1961. PDP – 1 was quickly followed by a series of other PDPs. In the meantime, one of the computer scientists at Bell Labs who had worked on the *MULTICS* project, Ken Thompson, subsequently found a small PDP-7 minicomputer that no one was using and set out to write a stripped-down, one-user version of MULTICS. This work later developed into the **UNIX** operating system, which became popular in the academic world, with government agencies, and with many companies.

The Fourth Generation (1980Present): *Personal Computers*

With the development of LSI (Large Scale Integration) circuits, chips containing thousands of transistors on a square centimeter of silicon, the age of the microprocessor-based personal computer dawned. In terms of architecture, personal computers (initially called microcomputers) were not all that different from minicomputers of the PDP-11 class, but in terms of price they certainly were different. The minicomputer made it possible for a department in a company or university to have its own computer. The microcomputer made it possible for an individual to have his or her own computer.

There were several families of microcomputers. Intel came out with the 8080, the first general-purpose 8-bit microprocessor, in 1974. Then a number of companies produced complete systems using the 8080 (or the compatible Zilog Z80). An operating system named from *CP/M (Control Program for Microcomputers)* from a company called Digital Research was widely used with these. Many application programs were written to run on CP/M, and it dominated the personal computing world for about 5 years. As Intel came out with the 8086 (a 16-bit system.), Microsoft offered an operating system referred to as *MS-DOS (MicroSoft Disk Operating System)* which quickly came to dominate the IBM PC market.

CP/M and MS-DOS were all command-line systems: users typed commands at the keyboard. Years earlier, Doug Engelbart at Stanford Research Institute had invented the GUI (Graphical User Interface), complete with windows, icons, menus, and mouse. Apple saw the possibility of a truly user-friendly personal computer and the Apple Macintosh (the first commercial GUI operating system) was announced in early 1984. To compete with the Macintosh, Microsoft invented Windows.

The other major contender in the personal computer world is UNIX. UNIX is strongest on workstations and other high-end computers, such as network servers. It is especially popular on

machines powered by high-performance RISC chips. On Pentium-based computers, **Linux** is becoming a popular alternative to Windows for students and increasingly many corporate users.

An interesting development that began taking place during the mid-1980s is the growth of networks of personal computers running *network operating systems* and *distributed operating systems*. In a network operating system, each machine runs its own local operating system and has its own local user (or users). Basically, the machines are independent of one another.

Network operating systems are not fundamentally different from single-processor operating systems. They obviously need a network interface controller and some low-level software to drive it, as well as programs to achieve remote login and remote file access, but these additions do not change the essential structure of the operating system.

A distributed operating system, in contrast, makes the interconnection of computers appear to its users as a traditional uniprocessor system. Users should not be aware of where their programs are being run or where their files are located; that should all be handled automatically and efficiently by the operating system. True distributed operating systems require more than just adding a little code to a uniprocessor operating system, because distributed and centralized systems differ in critical ways. Distributed systems, for example, often allow applications to run on several processors at the same time, thus requiring more complex processor scheduling algorithms in order to optimize the amount of parallelism. Communication delays within the network often mean that these (and other) algorithms must run with incomplete, outdated, or even incorrect information. This situation is radically different from a single-processor system in which the operating system has complete information about the system state.

Types of Operating Systems

All of the history and development, discussed above, has left us with a wide variety of operating systems, not all of which are widely known. In this section we will briefly touch upon some of them.

Mainframe Operating Systems

At the high end are the operating systems for the mainframes, those room-sized computers still found in major corporate data centers. These computers distinguish themselves from personal computers in terms of their I/O capacity. A mainframe with 1000 disks and thousands of gigabytes of data is not unusual. Mainframes are also making something of a comeback as high-end Web servers, servers for large-scale electronic commerce sites, and servers for business-to-business transactions.

The operating systems for mainframes are heavily oriented toward processing many jobs at once, most of which need prodigious amounts of I/O. They typically offer three kinds of services: *batch, transaction processing, and timesharing*.

- A batch system is one that processes routine jobs without any interactive user present. Claims processing in an insurance company or sales reporting for a chain of stores is typically done in batch mode.

- Transaction processing systems handle large numbers of small requests, for example, check processing at a bank or airline reservations. Each unit of work is small, but the system must handle hundreds or thousands per second.
- Timesharing systems allow multiple remote users to run jobs on the computer at once, such as querying a big database.

These functions are closely related and mainframe operating systems often perform all of them. Prominent mainframe operating systems include older versions like *Burroughs MCP*, *IBM OS/360*, *IBM CP-67*, *UNIVAC EXEC 8* and contemporary ones like *Unisys Clearpath/MCP*, *IBM z/OS*, *IBM z/VM*, and *Clearpath IX*.

Server Operating Systems

One level down are the server operating systems. They run on servers, which are very large personal computers, workstations, or even mainframes. They serve multiple users at once over a network and allow the users to share hardware and software resources. Servers can provide print service, file service, or Web service. Internet providers run many server machines to support their customers and Web sites use servers to store the Web pages and handle the incoming requests. Typical server operating systems are UNIX, Microsoft Windows Server products (2000, 2003, 2008), Mac OS X server and Linux.

Multiprocessor Operating Systems

An increasingly common way to get major-league computing power is to connect multiple CPUs into a single system. Depending on precisely how they are connected and what is shared, these systems are called *parallel computers*, *multicomputers*, or *multiprocessors*. They need special operating systems, but often these are variations on the server operating systems, with special features for communication and connectivity.

Personal Computer Operating Systems

These are category of operating systems whose major objective is to maximize user convenience and responsiveness instead of maximizing CPU and peripheral utilization. They provide a good interface to a single user and are widely used for text processing, spreadsheets management, entertainment and Internet access. Common examples are the various Windows operating systems (Windows, 98, 2000, XP, Vista), Mac OS X, and Linux.

Real-Time Operating Systems

Another type of operating system is the real-time system. These systems are characterized by having time as a key parameter. For example, in industrial process control systems, real-time computers have to collect data about the production process and use it to control machines in the factory. Often there are hard deadlines that must be met. For example, if a car is moving down an assembly line, certain actions must take place at certain instants of time, if a welding robot welds too early or too late, the car will be ruined. If the action absolutely must occur at a certain moment (or within a certain range), we have a ***hard real-time system***. Another kind of real-time system is a ***soft real-time system***, in which missing an occasional deadline is acceptable. Digital audio or multimedia systems fall in this category. VxWorks , QNX , eCos, FreeRTOS, OSEK, ChibiOS/RT, RT-11 and RTLinux are well-known real-time operating systems.

Embedded Operating Systems

Embedded operating systems are operating systems for embedded computer systems. These operating systems are designed to be very compact and efficient, forsaking many functions that non-embedded computer operating systems provide, and which may not be used by the specialized applications they run. They are frequently also real-time operating systems. Embedded systems run on the computers that control devices that are not generally thought of as computers, such as TV sets, microwave ovens, and mobile telephones. These often have some characteristics of real-time systems but also have size, memory, and power restrictions that make them special. Examples of such operating systems include PalmOS, Windows CE, A/ROSE, Embedded Linux, polyBSD (embedded NetBSD), Inferno, ROM-DOS, and Windows XP Embedded

Smart Card Operating Systems

These are the smallest operating systems run on smart cards, which are credit card-sized devices containing a CPU chip. They have very severe processing power and memory constraints. Some of them can handle only a single function, such as electronic payments, but others can handle multiple functions on the same smart card. Currently, the most notable smart card operating systems on the market are *JavaCard*, for program development using Java, and *MULTOS* which is the first open, high security, multi-application operating system for smart cards.

Computer Hardware

As we have discussed earlier, an operating system is intimately tied to the hardware of the computer it runs on. It extends the computer's instruction set and manages its resources. To work well, it must know a great deal about the hardware, at least, about how the hardware appears to the programmer.

Conceptually, a simple personal computer can be abstracted to a model resembling that of Figure 3. The CPU, memory, and I/O devices are all connected by a system bus and communicate with one another over it. Modern personal computers have a more complicated structure, involving multiple buses. But for the time being, this model will be sufficient. In the following sections, we will briefly review these components and examine some of the hardware issues that are of concern to operating system designers.

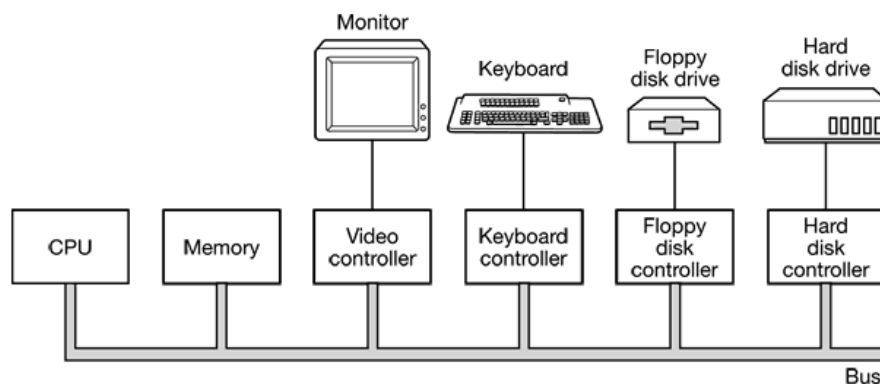


Figure 3 Some of the components of a simple personal computer.

Processors

The “brain” of the computer is the CPU. It fetches instructions from memory and executes them. The basic cycle of every CPU is to

- Fetch the first instruction from memory
- Decode it to determine its type and operands
- Execute it, and then fetch, decode, and execute subsequent instructions.

Each CPU has a specific set of instructions that it can execute. Thus a Pentium cannot execute SPARC programs and a SPARC cannot execute Pentium programs. Because accessing memory to get an instruction or data word takes much longer than executing an instruction, all CPUs contain some *registers* inside to hold key variables and temporary results. Thus the instruction set generally contains instructions to load a word from memory into a register, and store a word from a register into memory.

In addition to the general registers used to hold variables and temporary results, most computers have several special registers that are visible to the programmer. One of these is the program counter, which contains the memory address of the next instruction to be fetched. After that instruction has been fetched, the program counter is updated to point to its successor.

Another register is the stack pointer, which points to the top of the current stack in memory. The stack contains one frame for each procedure that has been entered but not yet exited. A procedure’s stack frame holds those input parameters, local variables, and temporary variables that are not kept in registers.

Yet another register is the Program Status Word (PSW). This register contains the condition code bits, which are set by comparison instructions, the CPU priority, the mode (user or kernel), and various other control bits. User programs may normally read the entire PSW but typically may write only some of its fields. The PSW plays an important role in system calls and I/O.

The operating system must be aware of all the registers. When time multiplexing the CPU, the operating system will often stop the running program to (re)start another one. Every time it stops a running program, the operating system must save all the registers so they can be restored when the program runs later.

To improve performance, CPU designers have long abandoned the simple model of fetching, decoding, and executing one instruction at a time. Many modern CPUs have facilities for executing more than one instruction at the same time. For example, a CPU might have separate fetch, decode, and execute units, so that while it was executing instruction n , it could also be decoding instruction $n + 1$ and fetching instruction $n + 2$. Such an organization is called a **pipeline** and is illustrated in Figure 4a for a pipeline with three stages.

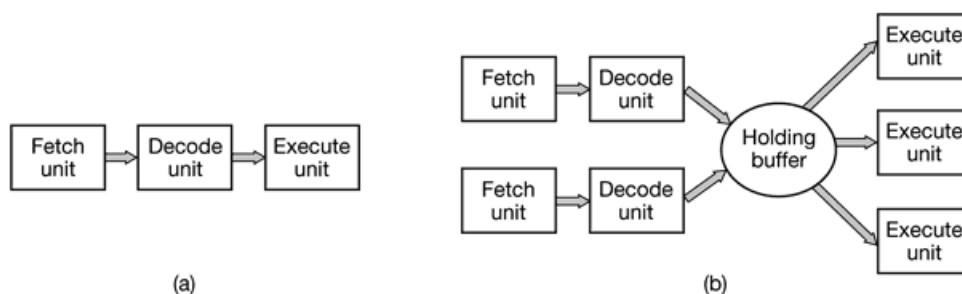


Figure 4 (a) A three-stage pipeline (b) A superscalar CPU.

Even more advanced than a pipeline design is a superscalar CPU, shown in Figure 4(b). In this design, multiple execution units are present, for example, one for integer arithmetic, one for floating-point arithmetic, and one for Boolean operations. Two or more instructions are fetched at once, decoded, and dumped into a holding buffer until they can be executed. As soon as an execution unit is free, it looks in the holding buffer to see if there is an instruction it can handle, and if so, it removes the instruction from the buffer and executes it.

An implication of this design is that program instructions are often executed out of order. For the most part, it is up to the hardware to make sure the result produced is the same one a sequential implementation would have produced, but an annoying amount of the complexity is foisted onto the operating system too.

Most CPUs, except very simple ones used in embedded systems, have two modes, **kernel mode** and **user mode**. Usually a bit in the PSW controls the mode. When running in kernel mode, the CPU can execute every instruction in its instruction set and use every feature of the hardware. The operating system runs in kernel mode, giving it access to the complete hardware. In contrast, user programs run in user mode, which permits only a subset of the instructions to be executed and a subset of the features to be accessed. Generally, all instructions involving I/O and memory protection are disallowed in user mode. Setting the PSW mode bit to kernel mode is also forbidden, of course.

To obtain services from the operating system, a user program must make a **system call**, which traps into the kernel and invokes the operating system. The TRAP instruction switches from user mode to kernel mode and starts the operating system. When the work has been completed, control is returned to the user program at the instruction following the system call.

Memory

The second major component in any computer is the memory. Ideally, a memory should be extremely fast (faster than executing an instruction so the CPU is not held up by the memory), abundantly large, and cheap. No current technology satisfies all of these goals, so a different approach is taken. The memory system is constructed as a hierarchy of layers, as shown in Figure 5. The top layer consists of the registers internal to the CPU. They are made of the same material as the CPU and are thus just as fast as the CPU. Consequently, there is no delay in accessing them. The maximum storage capacity available in them is typically 32 x 32-bits on a

32-bit CPU and 64 x 64-bits on a 64-bit CPU(less than 1 KB in both cases). Programs must manage the registers (i.e., decide what to keep in them) themselves, in software.

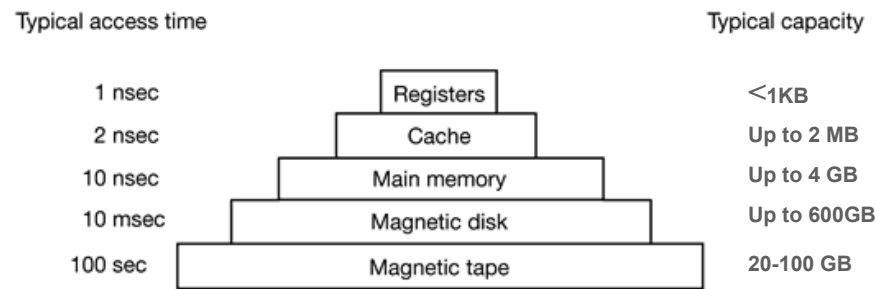


Figure 5 A typical memory hierarchy(the numbers are for PCs and very rough approximations)

Next comes the cache memory, which is a small memory on or close to the CPU chip and made faster than the much larger main memory. Most CPUs since the 1980s have used one or more caches, and modern general-purpose CPUs inside personal computers may have as many as half a dozen, each specialized to a different part of the problem of executing programs. These are organized in levels where each one is slower and bigger than the one before it.

Main memory is divided up into cache lines, typically 64 bytes, with addresses 0 to 63 in cache line 0, addresses 64 to 127 in cache line 1, and so on. The most heavily used cache lines are kept in a high-speed cache located inside or very close to the CPU. When the program needs to read a memory word, the cache hardware checks to see if the line needed is in the cache. If it is, called **a cache hit**, the request is satisfied from the cache and no memory request is sent over the bus to the main memory. If it is not found, called **a cache miss**, we have to go to memory, with a substantial time penalty. Cache memory is limited in size due to its high cost.

Main memory comes next. This is the workhorse of the memory system. Main memory is often called RAM (Random Access Memory). Currently, memories are hundreds to thousands of megabytes and growing rapidly. All CPU requests that cannot be satisfied out of the cache go to main memory.

Next in the hierarchy is magnetic disk (hard disk). Disk storage is many orders of magnitude cheaper than RAM per bit and often many orders of magnitude larger as well. The only problem is that the time to randomly access data on it is many orders of magnitude slower. This low speed is due to the fact that a disk is a mechanical device, as shown in Figure 6.

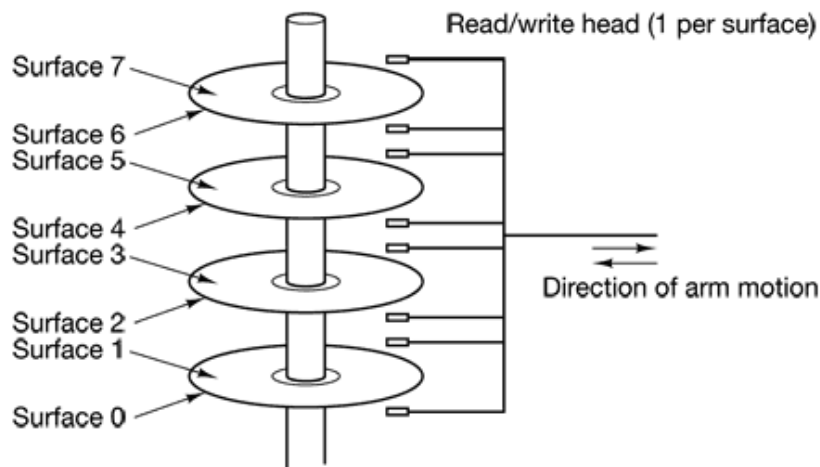


Figure 6 Structure of a hard disk drive

The final layer in the memory hierarchy is magnetic tape. This medium is often used as a backup for disk storage and for holding very large data sets. To access a tape, it must first be put into a tape reader (tape drive) and then the tape may have to be spooled forward to get to the requested block. All in all, this could take minutes. The big plus of tape is that it is exceedingly cheap per bit and removable, which is important for backup tapes that must be stored off-site in order to survive fires, floods, earthquakes, etc.

The memory hierarchy we have discussed is typical, but some installations do not have all the layers or have a few different ones (such as optical disk). Still, in all of them, as one goes down the hierarchy, the random access time increases dramatically, the capacity increases equally dramatically, and the cost per bit drops enormously. Consequently, it is likely that memory hierarchies will be around for years to come.

I/O Devices

Memory is not the only resource that the operating system must manage. I/O devices also interact heavily with the operating system. I/O devices generally consist of two parts: a controller and the device itself. The controller is a chip or a set of chips on a plug-in board that physically controls the device. It accepts commands from the operating system, for example, to read data from the device, and carries them out.

In many cases, the actual control of the device is very complicated and detailed, so it is the job of the controller to present a simpler interface to the operating system. For example, a disk controller might accept a command to read sector 11,206 from disk 2. The controller then has to convert this linear sector number to a cylinder, sector, and head. This conversion may be complicated by the fact that outer cylinders have more sectors than inner ones and that some bad sectors have been remapped onto other ones. Then the controller has to determine which cylinder the disk arm is on and give it a sequence of pulses to move in or out the requisite number of

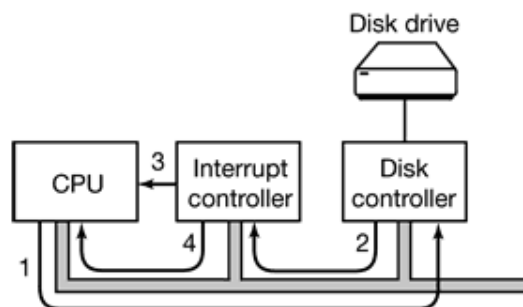
cylinders. It has to wait until the proper sector has rotated under the head and then start reading and storing the bits as they come off the drive, removing the preamble and computing the checksum. Finally, it has to assemble the incoming bits into words and store them in memory. To do all this work, controllers often contain small embedded computers that are programmed to do their work.

The other piece is the actual device itself. Devices have fairly simple interfaces, both because they cannot do much and to make them standard. Since the actual device interface is hidden behind the controller, all that the operating system sees is the interface to the controller which may be quite different from the interface to the device.

Because each type of controller is different, different software is needed to control each one. The software that talks to a controller, giving it commands and accepting responses, is called a **device driver**. Each controller manufacturer has to supply a driver for each operating system it supports. Thus a scanner may come with drivers for Windows 98, 2000, XP and UNIX, for example.

Input and output can be done in three different ways.

- In the simplest method, a user program issues a system call, which the kernel then translates into a procedure call to the appropriate driver. The driver then starts the I/O and sits in a tight loop continuously polling the device to see if it is done (usually there is some bit that indicates that the device is still busy). When the I/O has completed, the driver puts the data where they are needed (if any), and returns. The operating system then returns control to the caller. This method is called **Programmed I/O** and has the disadvantage of tying up the CPU polling the device until it is finished.
- The second method is for the driver to start the device and ask it to give an interrupt when it is finished. The operating system then blocks the caller if need be and looks for other work to do. When the controller detects the end of the transfer, it generates an interrupt to signal completion. This kind of I/O is called **Interrupt driven I/O** and the operation is depicted in Figure 7



Step 1 - the driver tells the controller what to do by writing into its device registers. The controller then starts the device.

Step 2 - The controller has finished reading or writing the number of bytes it has been told to transfer and it signals the interrupt controller chip using certain bus lines.

Step 3- Interrupt controller asserts the interrupt to a pin on the CPU chip

Step 4, the interrupt controller puts the number of the device on the bus so the CPU can read it and know which device has just finished (because many devices may be running at the same time).

Figure 7 The steps in starting an I/O device and getting an interrupt

- The third method for doing I/O makes use of a special DMA (Direct Memory Access) chip that can control the flow of bits between memory and some controller without constant CPU intervention. The CPU sets up the DMA chip telling it how many bytes to transfer, the device and memory addresses involved, and the direction, and lets it go. When the DMA chip is done, it causes an interrupt, which is handled as described above.

Buses

The organization of Figure 3 was used on minicomputers for years and also on the original IBM PC. However, as processors and memories got faster, the ability of a single bus to handle all the traffic was strained to the breaking point. As a result, additional buses were added, both for faster I/O devices and for CPU to memory traffic. As a consequence of this evolution, a large Pentium system currently looks like the one shown in Figure 8.

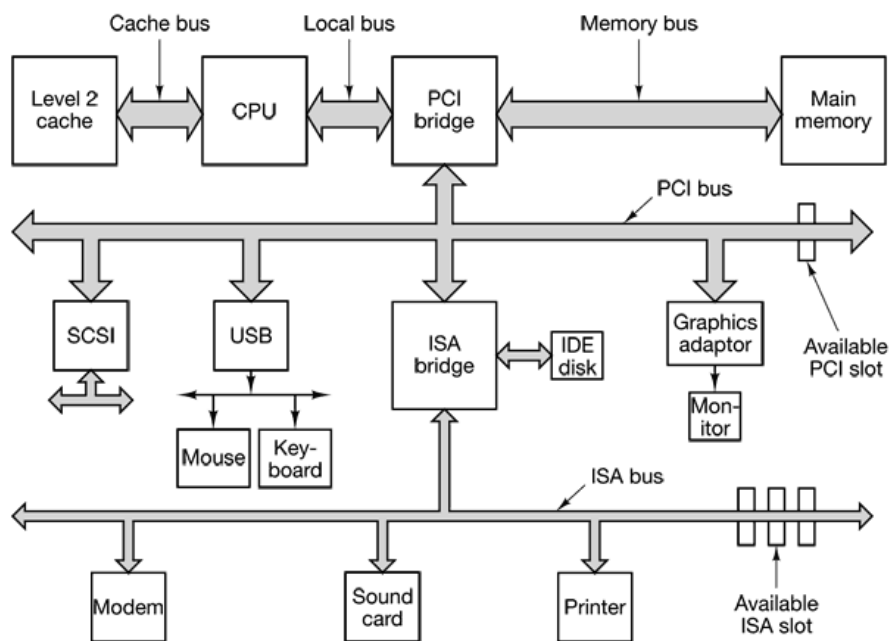


Figure 8 The structure of a large Pentium system

This system has eight buses (cache, local, memory, PCI, SCSI, USB, IDE, and ISA), each with a different transfer rate and function. The two main buses are the original IBM PC ISA (Industry Standard Architecture) bus and its successor, the PCI (Peripheral Component Interconnect) bus. The ISA bus runs at 8.33 MHz and can transfer 2 bytes at once, for a maximum speed of 16.67 MB/sec. It is included for backward compatibility with old and slow I/O cards. The PCI bus was invented by Intel as a successor to the ISA bus. It can run at 66 MHz and transfer 8 bytes at a time, for a data rate of 528 MB/sec. Most high-speed I/O devices use the PCI bus now. Even some non-Intel computers use the PCI bus due to the large number of I/O cards available for it.

In this configuration, the CPU talks to the PCI bridge chip over the local bus, and the PCI bridge chip talks to the memory over a dedicated memory bus, often running at 100-533 MHz. Pentium

systems have a level-1 cache on chip and a much larger level-2 cache off chip, connected to the CPU by the cache bus.

In addition, this system contains three specialized buses: IDE, USB, and SCSI. The IDE bus is for attaching peripheral devices such as disks and CD-ROMs to the system. The IDE bus is now standard on nearly all Pentium-based systems for the hard disk and often the CD-ROM.

The USB (Universal Serial Bus) was invented to attach all the slow I/O devices, such as the keyboard and mouse, to the computer. It uses a small four-wire connector, two of which supply electrical power to the USB devices. USB is a centralized bus in which a root device polls the I/O devices every 1 msec to see if they have any traffic. The current standard can handle an aggregate load of 60 MB/sec. All the USB devices share a single USB device driver, making it unnecessary to install a new driver for each new USB device. Consequently, USB devices can be added to the computer without the need to reboot.

The SCSI (Small Computer System Interface) bus is a high-performance bus intended for fast disks, scanners, and other devices needing considerable bandwidth. It can run at up to 160 MB/sec. It has been present on Macintosh systems since they were invented and is also popular on UNIX and some Intel-based systems.

A new specialized bus known as Accelerated Graphics port (AGP) which is dedicated to video information and capable of handling up to 2,133 MB/s is also of part of contemporary bus systems.

To work in an environment such as that of Fig. 8, the operating system has to know what is out there and configure it.

Functions of Operating Systems

All operating systems have certain basic tasks such as processes management, memory management, I/O management file management and security. In this section, we will look at some of these basic concepts very briefly.

Process Management

A key concept in all operating systems is the process. *A process is basically a program in execution.* Associated with each process are:

- Its address space, a list of memory locations from some minimum (usually 0) to some maximum, which the process can read and write. This space contains the *executable program, the program's data, and its stack.*
- Some set of registers, including the *program counter, stack pointer, and other hardware registers*, and all the other information needed to run the program.

In order to get a good intuitive feel for a process let's think about timesharing systems. In these systems, periodically, the operating system decides to stop running one process and start running another, for example, because the first one has had more than its share of CPU time in the past second. When a process is suspended temporarily like this, it must later be restarted in exactly the same state it had when it was stopped. This means that all information about the process must

be explicitly saved somewhere during the suspension. For example, the process may have several files open for reading at once. Associated with each of these files is a pointer giving the current position (i.e., the number of the byte or record to be read next). When a process is temporarily suspended, all these pointers must be saved so that a read call executed after the process is restarted will read the proper data. In many operating systems, all the information about each process, other than the contents of its own address space, is stored in an operating system table called the **process table**, which is an array (or linked list) of structures, one for each process currently in existence. Thus, a (suspended) process consists of its address space, usually called the core, and its process table entry, which contains its registers, among other things.

The operating system is responsible for providing key process management system calls to deal with the *creation*, *inter-process communication*, *synchronization*, and *termination* processes.

Memory Management

Every computer has some main memory that it uses to hold executing programs. In a very simple operating system, only one program at a time is in memory. To run a second program, the first one has to be removed and the second one placed in memory.

More sophisticated operating systems allow multiple programs to be in memory at the same time. To keep them from interfering with one another (and with the operating system), some kind of protection mechanism is needed. While this mechanism has to be in the hardware, it is controlled by the operating system.

The above viewpoint is concerned with *managing* and *protecting* the computer's main memory. A different, but equally important memory-related issue is managing the address space of the processes. Normally, each process has some set of addresses it can use, typically running from 0 up to some maximum. In the simplest case, the maximum amount of address space a process has is less than the main memory. In this way, a process can fill up its address space and there will be enough room in main memory to hold it all.

However, on many computers addresses are 32 or 64 bits, giving an address space of 2^{32} or 2^{64} bytes, respectively. If a process has more address space than the computer's main memory and the process wants to use it all, a technique called **virtual memory** is used. In this technique, the operating system keeps part of the address space in main memory and part on disk and shuttles pieces back and forth between them as needed. This is one of the important operating system functions.

Input/Output Management

All computers have physical devices for acquiring input and producing output. Consequently, every operating system has an I/O subsystem for managing its I/O devices. Some of the I/O software is device independent, that is, applies to many or all I/O devices equally well. Other parts of it, such as device drivers, are specific to particular I/O devices.

File System Management

Another key concept supported by virtually all operating systems is the file system. As noted before, a major function of the operating system is to hide the peculiarities of the disks and other I/O devices and present the programmer with a nice, clean abstract model of device-independent files. System calls are obviously needed to create files, remove files, read files, and write files. To provide a place to keep files, most operating systems have the concept of a directory as a way of grouping files together. A student, for example, might have one directory for each course he is taking (for the programs needed for that course), another directory for his electronic mail, and still another directory for his World Wide Web home page. System calls are then needed to create and remove directories. Calls are also provided to put an existing file in a directory, and to remove a file from a directory. Directory entries may be either files or other directories. This model gives rise to a hierarchy—the file system—as shown in Figure 9.

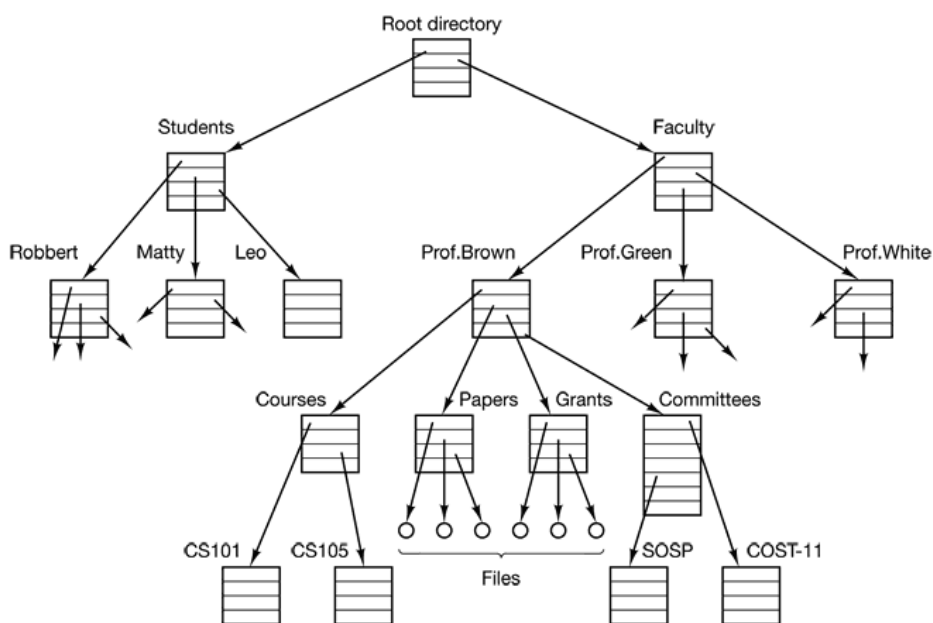


Figure 9. A file system for a university department

Security

Computers contain large amounts of information that users often want to keep confidential. This information may include electronic mail, business plans, tax returns, and much more. It is up to the operating system to manage the system security so that files, for example, are only accessible to authorized users. In addition to file protection, there are many other security issues. Protecting the system from unwanted intruders, both human and nonhuman (e.g., viruses) is one of them.

SYSTEM CALLS

The interface between the operating system and the user programs is defined by the set of system calls that the operating system provides. The system calls available in the interface vary from operating system to operating system (although the underlying concepts tend to be similar).

To really understand what operating systems do, let's examine this interface in UNIX closely. If a process is running a user program in user mode and needs a system service, such as reading data from a file, it has to execute a system call instruction to transfer control to the operating system. The operating system then figures out what the calling process wants by inspecting the parameters. Then it carries out the system call and returns control to the instruction following the system call. In a sense, making a system call is like making a special kind of procedure call, *only system calls enter the kernel and procedure calls do not*.

To make the system call mechanism clearer, let us take a quick look at the **read** system call. **read** has three parameters: the first one specifying the file, the second one pointing to the buffer, and the third one giving the number of bytes to read. Like nearly all system calls, it is invoked from C programs by calling a library procedure with the same name as the system call: **read**. A call from a C program might look like this:

```
count = read(fd, buffer, nbytes);
```

The system call (and the library procedure) return the number of bytes actually read in count. This value is normally the same as nbytes, but may be smaller, if, for example, end-of-file is encountered while reading. If the system call cannot be carried out, either due to an invalid parameter or a disk error, count is set to -1, and the error number is put in a global variable, `errno`. Programs should always check the results of a system call to see if an error occurred.

System calls are performed in a series of steps. For instance, if we examine the **read** call discussed above,

1. In preparation for calling the read library procedure, which actually makes the read system call, the calling program first pushes the parameters onto the stack, as shown in steps 1-3 in Fig. 10.
2. Then comes the actual call to the library procedure (step 4). This instruction is the normal procedure call instruction used to call all procedures. The library procedure, possibly written in assembly language, typically puts the system call number in a place where the operating system expects it, such as a register (step 5). Then it executes a TRAP instruction to switch from user mode to kernel mode and start execution at a fixed address within the kernel (step 6).
3. The kernel code that starts examines the system call number and then dispatches to the correct system call handler, usually via a table of pointers to system call handlers indexed on system call number (step 7). At that point the system call handler runs (step 8).
4. Once the system call handler has completed its work, control may be² returned to the user-space library procedure at the instruction following the TRAP instruction (step 9). This procedure then returns to the user program in the usual way procedure calls return (step 10).
5. To finish the job, the user program has to clean up the stack, as it does after any procedure call (step 11).

² Sometimes, the system call may block the caller, preventing it from continuing. For example, if it is trying to read from the keyboard and nothing has been typed yet, the caller has to be blocked. In this case, the operating system will look around to see if some other process can be run next. Later, when the desired input is available, this process will get the attention of the system and steps 9-11 will occur.

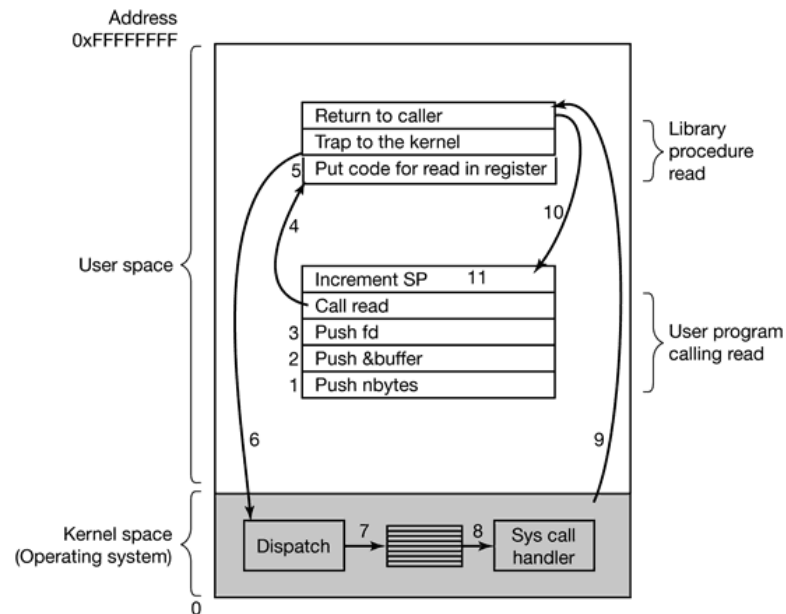


Figure 10 - Steps in making the system call `read(id, buffer, nbytes)`

Operating System Structure

Now that we have seen what operating systems look like on the outside (i.e., the programmer's interface), it is time to take a look inside. In the following sections, we will examine five different structures that have been tried, in order to get some idea of the spectrum of possibilities. These are by no means exhaustive, but they give an idea of some designs that have been tried in practice. The five designs are monolithic systems, layered systems, virtual machines, exokernels, and client-server systems.

Monolithic Systems

By far the most common organization, this approach might well be subtitled "The Big Mess." In this structure, the operating system is written as a collection of procedures, each of which can call any of the other ones whenever it needs to. When this technique is used, each procedure in the system has a well-defined interface in terms of parameters and results, and each one is free to call any other one, if the latter provides some useful computation that the former needs.

To construct the actual object program of the operating system when this approach is used, one first compiles all the individual procedures, or files containing the procedures, and then binds them all together into a single object file using the system linker. In terms of information hiding, there is essentially none—every procedure is visible to every other procedure.

Layered Systems

The other approach to organize the operating system as a hierarchy of layers, each one constructed upon the one below it. The first system constructed in this way was the **THE** system built at the Technische Hogeschool Eindhoven in the Netherlands by E. W. Dijkstra (1968) and his students. The **THE** system was a simple batch system for a Dutch computer, the Electrologica X8.

The system had 6 layers, as shown in Table 1.

Table 1 Structure of the THE operating system

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

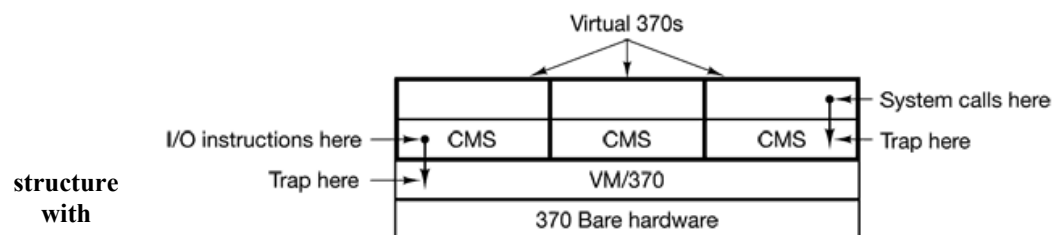
- Layer 0 dealt with allocation of the processor, switching between processes when interrupts occurred or timers expired. Above layer 0, the system consisted of sequential processes, each of which could be programmed without having to worry about the fact that multiple processes were running on a single processor. In other words, layer 0 provided the basic multiprogramming of the CPU.
- Layer 1 did the memory management. It allocated space for processes in main memory and on a 512K word drum used for holding parts of processes (pages) for which there was no room in main memory. Above layer 1, processes did not have to worry about whether they were in memory or on the drum; the layer 1 software took care of making sure pages were brought into memory whenever they were needed.
- Layer 2 handled communication between each process and the operator console. Above this layer each process effectively had its own operator console.
- Layer 3 took care of managing the I/O devices and buffering the information streams to and from them. Above layer 3 each process could deal with abstract I/O devices with nice properties, instead of real devices with many peculiarities.
- Layer 4 was where the user programs were found. They did not have to worry about process, memory, console, or I/O management. The system operator process was located in layer 5.

A further generalization of the layering concept was present in the **MULTICS** system. Instead of layers, MULTICS was described as having a series of concentric rings, with the inner ones being more privileged than the outer ones (which is effectively the same thing). When a procedure in an outer ring wanted to call a procedure in an inner ring, it had to make the equivalent of a system call, whose parameters were carefully checked for validity before allowing the call to proceed.

Virtual Machines

An IBM product originally called CP/CMS and later renamed VM/370 was the first operating system to exhibit the concept of virtual machines. The essence of VM/370 is to completely separate these two functions. a timesharing system provides - multiprogramming and an extended machine with a more convenient interface than the bare hardware

The heart of the system, known as the virtual machine monitor, runs on the bare hardware and does the multiprogramming, providing not one, but several virtual machines to the next layer up, as shown in Figure 11. However, unlike all other operating systems, these virtual machines are not extended machines, with files and other nice features. Instead, they are exact copies of the bare hardware, including kernel/user mode, I/O, interrupts, and everything else the real machine has.



Because each virtual machine is identical to the true hardware, each one can run any operating system that will run directly on the bare hardware. Different virtual machines can, and frequently do, run different operating systems. Some run one of the descendants of OS/360 for batch or transaction processing, while other ones run a single-user, interactive system called CMS (Conversational Monitor System) for interactive timesharing users.

When a CMS program executes a system call, the call is trapped to the operating system in its own virtual machine, not to VM/370, just as it would if it were running on a real machine instead of a virtual one. CMS then issues the normal hardware I/O instructions for reading its virtual disk or whatever is needed to carry out the call. These I/O instructions are trapped by VM/370, which then performs them as part of its simulation of the real hardware. By completely separating the functions of multiprogramming and providing an extended machine, each of the pieces can be much simpler, more flexible, and easier to maintain.

The idea of a virtual machine is heavily used nowadays in a different context: running old MS-DOS programs on a Pentium (or other 32-bit Intel CPU). When designing the Pentium and its software, both Intel and Microsoft realized that there would be a big demand for running old software on new hardware. For this reason, Intel provided a virtual 8086 mode on the Pentium. In this mode, the machine acts like an 8086 (which is identical to an 8088 from a software point of view), including 16-bit addressing with a 1-MB limit. This mode is used by Windows and other operating systems for running MS-DOS programs.

Exokernels

With VM/370, each user process gets an exact copy of the actual computer. With virtual 8086 mode on the Pentium, each user process gets an exact copy of a different computer. Going one step further, researchers at M.I.T. have built a system that gives each user a clone of the actual computer, but with a subset of the resources. Thus one virtual machine might get disk blocks 0 to 1023, the next one might get blocks 1024 to 2047, and so on.

At the bottom layer, running in kernel mode, is a program called the exokernel. Its job is to allocate resources to virtual machines and then check attempts to use them to make sure no machine is trying to use somebody else's resources. Each user-level virtual machine can run its own operating system, as on VM/370 and the Pentium virtual 8086s, except that each one is restricted to using only the resources it has asked for and been allocated.

The advantage of the exokernel scheme is that it saves a layer of mapping. In the other designs, each virtual machine thinks it has its own disk, with blocks running from 0 to some maximum, so the virtual machine monitor must maintain tables to remap disk addresses (and all other resources). With the exokernel, this remapping is not needed. The exokernel need only keep track of which virtual machine has been assigned which resource. This method still has the advantage of separating the multiprogramming (in the exokernel) from the user operating system code (in user space), but with less overhead, since all the exokernel has to do is keep the virtual machines out of each other's hair.

Client-Server Model

VM/370 gains much in simplicity by moving a large part of the traditional operating system code (implementing the extended machine) into a higher layer, CMS. Nevertheless, VM/370 itself is still a complex program because simulating a number of virtual 370s in their entirety is not that simple (especially if you want to do it reasonably efficiently).

A trend in modern operating systems is to take the idea of moving code up into higher layers even further and remove as much as possible from kernel mode, leaving a minimal microkernel. The usual approach is to implement most of the operating system in user processes. To request a service, such as reading a block of a file, a user process (now known as the client process) sends the request to a server process, which then does the work and sends back the answer.

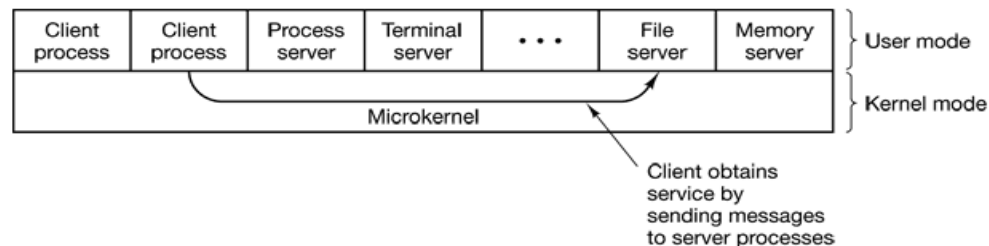


Figure 12 The client-server model

In this model, all the kernel does is handle the communication between clients and servers. By splitting the operating system up into parts, each of which only handles one facet of the system, such as file service, process service, terminal service, or memory service, each part becomes small and manageable. Furthermore, because all the servers run as user-mode processes, and not in kernel mode, they do not have direct access to the hardware. As a consequence, if a bug in the file server is triggered, the file service may crash, but this will not usually bring the whole machine down.